

PROGRAMMING II

ARRAYS AND ENUMERATIVE TYPES

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

ARRAYS



ARRAYS IN JAVA

An **array** is an ordered sequence of variables of the **same type** which are accessed through an index (a natural number)

- ▶ Can contain both primitive types or object references
- ▶ Cannot contain object values
- ▶ Array index in Java start from 0 *// as it always should be*

Array dimension can be defined at **run-time**, during object creation

- ▶ But cannot change afterwards

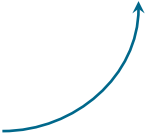
ARRAYS IN JAVA (CONT'D)

Array in Java are special objects (allocated in the heap)

- ▶ Array declaration allocates a **reference**

`int arr[];` ----- ^{equivalent} `int[] arr;`

No need to specify the dimension, since we are declaring a reference



Reference to an array of `int` (initialized to `null`)



ARRAY CREATION

With the `new` operator

```
float[] arr = new float[10]; // allocates an array of 10 elements      1
int size = 7;                // elements take the default value        2
float[] arr = new float[size]; // array dimension given at runtime     3
```

With static initialization

```
int[] primes = new int[]{2, 3, 5}; // dimension automatically inferred 1
int[] primes = {2, 3, 5};           // shorter notation                 2
Car[] cars = {new Car("red"), myCar}; // values or references allowed 3
```

ARRAY ACCESS

Array elements are read with brackets []

- ▶ Array bounds are checked at runtime

```
int[] primes = new int[]{2, 3, 5};           1
System.out.println(primes[1]); // the output will be '3' 2
System.out.println(primes[6]); // the JVM will throw an exception 3
```

Array elements are written with brackets []

```
Car[] cars = new Car[3];                      1
cars[1] = new Car("red"); // cars points to [null, Car@25c7f37d, null] 2
```

ARRAY OPERATIONS

Each array comes with a (pseudo) field `length`

```
int[] primes = new int[]{2, 3, 5, 7};           1  
System.out.println(primes.length); // the output will be '4' 2
```

If the array is not initialized or `null`, the `length` is not accessible

```
int[] primes = null;                             1  
System.out.println(primes.length); // the JVM will throw an exception 2
```

ARRAY OPERATIONS (CONT'D)

As for any object, == is applied to the reference and not to array elements

- ▶ The equals() method does not work for arrays

```
int[] arr1 = {1,2,3}, arr2 = {1,2,3};           1
System.out.println(arr1 == arr2);                // the output is 'false' 2
System.out.println(arr1.equals(arr2));            // the output is 'false' 3
System.out.println(Arrays.equals(arr1, arr2));    // the output is 'true' 4
```

Use the Arrays.equals() method from the java.util package

- ▶ You need to import the class java.util.Arrays

ARRAY OPERATIONS (CONT'D)

The same goes for the `toString()` methods of arrays

- ▶ It prints the reference and not the array content

```
int[] arr = {1,2,3};                                1
System.out.println(arr.toString());                  // the output is '[I@6ce253f1' 2
System.out.println(Arrays.toString(arr));             // the output is '[1, 2, 3]' 3
```

Use the `Arrays.toString()` method from the `java.util` package

- ▶ You need to import the class `java.util.Arrays`

ENHANCED FOR LOOP

New loop construct available from Java 5

`for`(*Type var : IterExpr*)

where *IterExpr* can be

- An array or an iterable class

// we will see iterables later

```
for (String arg : args)           // print program arguments    1
    System.out.println(arg);      2
                                   3
for (int i = 0; i < args.length; i++) {    4
    String arg = args[i];           5
    System.out.println(arg);        // the two loops are equivalent 6
}                                    7
```

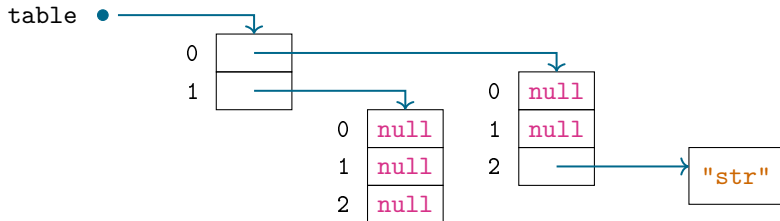
MULTIDIMENSIONAL ARRAY

Implemented as arrays of arrays

```
String[][] table = new String[2][3];  
table[0][2] = new String("str");
```

1

2



MULTIDIMENSIONAL ARRAY (CONT'D)

Rows are not stored in adjacent positions in memory

- ▶ They can be easily exchanged
- ▶ They can have different length

```
int[][] matrix = { {1,2,3}, {4,5,6}, {7,8,9} };           1
int[] temp = matrix[0];                                   2
matrix[0] = matrix[matrix.length-1];                      3
matrix[matrix.length-1] = temp; // first row swapped with the last 4
                                                                    5

int[][] pyramid new int[3][]; // pyramid is:              6
pyramid[0] = new int[1]; // [null]                        7
pyramid[1] = new int[2]; // [null, null]                  8
pyramid[2] = new int[3]; // [null, null, null]            9
```

COMPARISON AND STRING ISSUES AGAIN

Multidimensional arrays **should not** be compared

- ▶ By using `==`, `equals()` and `Arrays.equals()` too
- ▶ The same applies to `toString()` and `Arrays.toString()` methods

The programmer needs to loop over rows

```
int[][] matrix = { {1,2,3}, {4,5,6}, {7,8,9} };           1
System.out.println(Arrays.toString(matrix));               2
// the output is '[[I@53d8d10a, [I@e9e54c2, [I@65ab7765]]' 3
                                                            4
for (int[] row : matrix) {                                5
    System.out.println(Arrays.toString(row));              6
}                                                          7
// the output is '[1, 2, 3]'
//                  [4, 5, 6]
//                  [7, 8, 9]'
```

ENUMERATIVE TYPES



ENUMS

Enumerative types allow variables to range over a fixed set of constants

- ▶ Introduced in Java 5 with the keyword `enum`
- ▶ Variables can take only one of the enumerated values

```
public enum Direction {  
    NORTH, SOUTH, EAST, WEST  
}  
  
Direction move = Direction.WEST;
```

1
2
3
4
5

ENUMS (CONT'D)

Enumerative types can be declared

- ▶ Inside a class or in a separate file (as any other class)
- ▶ But not within a method

Enumerative types are not strings or numbers

- ▶ Special classes whose constructor cannot be invoked directly
- ▶ But they have a string and a numeric representation

```
public enum Direction { NORTH, SOUTH, EAST, WEST }      1
Direction move = Direction.WEST;                          2
                                                           3
System.out.println(move.name()); // the output is 'WEST'  4
System.out.println(move.ordinal()); // the output is '3'  5
```

ENUM COMPARISON

Enumerative types **can** be compared with `==` safely

- ▶ Enums ensure that only one instance of the constants exist in the JVM
- ▶ Actually, it is safer than `equals()`

```
public enum Direction { NORTH, SOUTH, EAST, WEST }           1
Direction move1 = Direction.WEST, move2 = null;               2
                                                                3
System.out.println(move1 == Direction.WEST); // the output is 'true' 4
System.out.println(move2 == Direction.WEST); // the output is 'false' 5
System.out.println(move2.equals(Direction.WEST)); // JVM exception    6
```

ENUM USAGE

Enums can be used in switch statements

```
public String moveDirectionAsString(Direction move) { 1
    switch (move) { 2
        case NORTH: return "↑"; 3
        case SOUTH: return "↓"; 4
        case EAST: return "→"; 5
        case WEST: return "←"; 6
    } 7
    return null; 8
} 9
10
11
```

ENUM USAGE (CONT'D)

Enums can be used in enhanced for loops

```
public void printDirections() {  
    for (Direction d : Direction.values()) {  
        System.out.println(d.name());  
    }  
}
```

1
2
3
4
5
6
7

Q + A

